



Integración de Bootstrap y FontAwesome en Plantillas HTML de Spring Boot

Introducción

En esta guía vamos a **integrar Bootstrap y FontAwesome** directamente en las plantillas HTML de nuestra aplicación Spring Boot.

El objetivo es que la aplicación tenga un **mejor diseño visual** utilizando estas tecnologías, además de preparar la base para aplicar estilos a nuestras páginas de login, index y formularios.

Hoy trabajaremos en:

- ✿ Agregar Bootstrap y FontAwesome al layout principal (`plantilla.html`).
- ✿ Configurar correctamente los links CSS y scripts JavaScript.
- ✿ Ajustar las plantillas `index.html`, `login.html` y `modificar.html` para que utilicen los fragmentos del layout.

☀️ Ajustar el archivo `SecurityConfig.java` para aceptar los archivos webjar como css, o archivos js (de JavaScript).

Todo esto nos permitirá que el estilo visual se aplique correctamente a toda la aplicación.

Desarrollo Paso a Paso

1. Modificar `plantilla.html` para agregar Bootstrap y FontAwesome

📁 Ruta del archivo:

`Controlclientes/src/main/resources/templates/layout/plantilla.html`

Código trabajado:

Agregamos en la sección `<head>` lo siguiente:

```
<!DOCTYPE html>
<html data-bs-theme="dark"
      xmlns:th="http://www.thymeleaf.org"
      xmlns:sec="http://www.thymeleaf.org/extras/spring-security">
  <head th:fragment="head">
    <title>Plantilla</title>
    <meta charset="UTF-8"/>
    <!-- CSS de Bootstrap -->
    <link rel="stylesheet" th:href="@{/webjars/bootstrap/css/bootstrap.min.css}"/>
    <!-- CSS de Font Awesome -->
    <link rel="stylesheet" th:href="@{/webjars/font-awesome/css/all.css}"/>
    <!-- JavaScript de Bootstrap (incluye Popper.js) -->
    <script th:src="@{/webjars/bootstrap/js/bootstrap.bundle.min.js}"></script>
  </head>
```

Explicación del código agregado

- ♦ Se crea un **fragmento head** para que las demás páginas puedan **reutilizar** este bloque.
- ♦ Se agregan los **estilos de Bootstrap** y los **iconos de FontAwesome** utilizando WebJars.
- ♦ Se agregan el **script de JavaScript** necesario para el funcionamiento dinámico de Bootstrap, como modales, tooltips, etc.

Así evitamos errores y mantenemos limpio el código.

2. Modificar `index.html` para utilizar el nuevo `head`

Ruta del archivo:

ControlClientes/src/main/resources/templates/index.html

Código trabajado:

Reemplazamos la sección `<head>` para usar el fragmento de plantilla:

```
<!DOCTYPE html>
<html data-bs-theme="dark"
      xmlns:th="http://www.thymeleaf.org"
      xmlns:sec="http://www.thymeleaf.org/extras/spring-security">
  <head th:replace="~{layout/plantilla :: head}">
    <title>Inicio</title>
    <meta charset="UTF-8">
  </head>
```

Explicación

Ahora `index.html` **importa automáticamente** todas las librerías necesarias desde el fragmento `head` definido en `plantilla.html`.

Así mantenemos consistencia visual y evitamos duplicar código.

3. Modificar `login.html` para utilizar el nuevo `head`

Ruta del archivo:

ControlClientes/src/main/resources/templates/login.html

Código trabajado:

Reemplazamos también el `<head>` de `login.html`:

```
<!DOCTYPE html>
<html data-bs-theme="dark"
      xmlns:th="http://www.thymeleaf.org">
  <head th:replace="~{layout/plantilla :: head}">
    <title>Login</title>
    <meta charset="UTF-8"/>
  </head>
```

Explicación

De esta forma, la página de inicio de sesión también tendrá disponibles los estilos de Bootstrap y los iconos de FontAwesome.

4. Modificar `modificar.html` para utilizar el nuevo `head`

Ruta del archivo:

`ControlClientes/src/main/resources/templates/modificar.html`

Código trabajado:

Reemplazamos también el `<head>` de `login.html`:

```
<!DOCTYPE html>
<html lang="es" data-bs-theme="dark"
      xmlns:th="http://www.thymeleaf.org">
  <head th:replace="~{layout/plantilla :: head}">
    <title>Datos de la Persona</title>
    <meta charset="UTF-8"/>
  </head>
```

Explicación

De esta forma, la página de inicio de sesión también tendrá disponibles los estilos de Bootstrap y los iconos de FontAwesome.

5. Modificar `SecurityConfig.java` para aceptar las peticiones de webjars

Ruta del archivo:

`ControlClientes/src/main/java/gm/web/SecurityConfig.java`

Código trabajado:

Agregamos el path de `/webjars**` para aceptar las peticiones de los estilos de Bootstrap y FontAwesome:

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    return http
        .authorizeHttpRequests(authorize -> authorize
            .requestMatchers("/webjars/**", "/resources/**", "/login").permitAll()
            .requestMatchers("/editar/**", "/agregar/**", "/eliminar/**").hasRole("ADMIN")
            .requestMatchers("/").hasAnyRole("USER", "ADMIN")
        )
    ;
}
```

```
        .anyRequest().authenticated()
    )
    .formLogin(form -> form
        .loginPage("/login")
        .defaultSuccessUrl("/", true)
        .permitAll()
    )
    .logout(logout -> logout
        .logoutUrl("/logout")
        .logoutSuccessUrl("/login?logout")
        .invalidateHttpSession(true)
        .deleteCookies("JSESSIONID")
    )
    .exceptionHandling(exception -> exception
        .accessDeniedPage("/errores/403")
    )
    .build();
}
```

Explicación

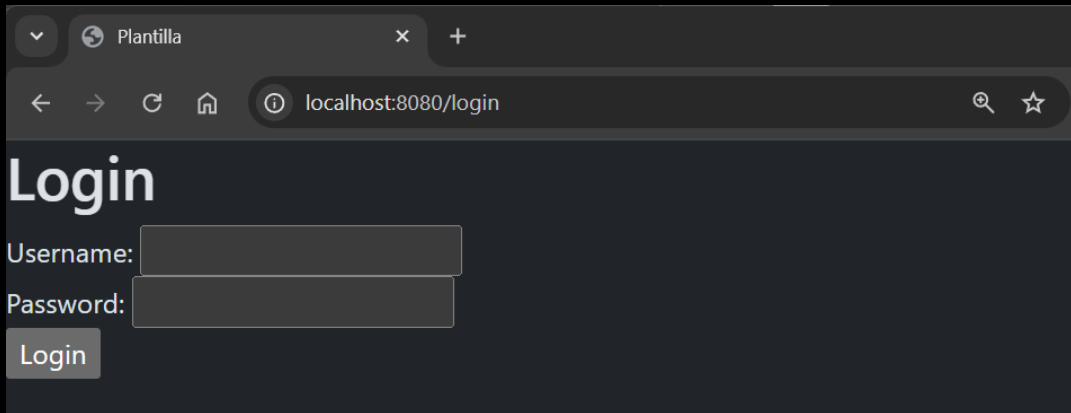
Agregamos `/webjars/**` para aceptar las peticiones de las rutas de los elementos de Bootstrap y FontAwesome, como son archivos `.css` o `.js`, de lo contrario no son rutas públicas y la seguridad de Spring no permitirá acceder a estos archivos, evitando desplegar los nuevos estilos.

4. Ejecutar la Aplicación

Ejecutamos la aplicación para probar:

- Abrimos el navegador en `http://localhost:8080`
- Observamos que ya se aplican **los estilos de Bootstrap y la nueva fuente**.
- Verificamos que los links de CSS y JavaScript se cargan correctamente (puedes hacer clic derecho → Ver código fuente).

Si ves cambios en la fuente y estilos más modernos, ¡todo está funcionando correctamente! 



Plantilla

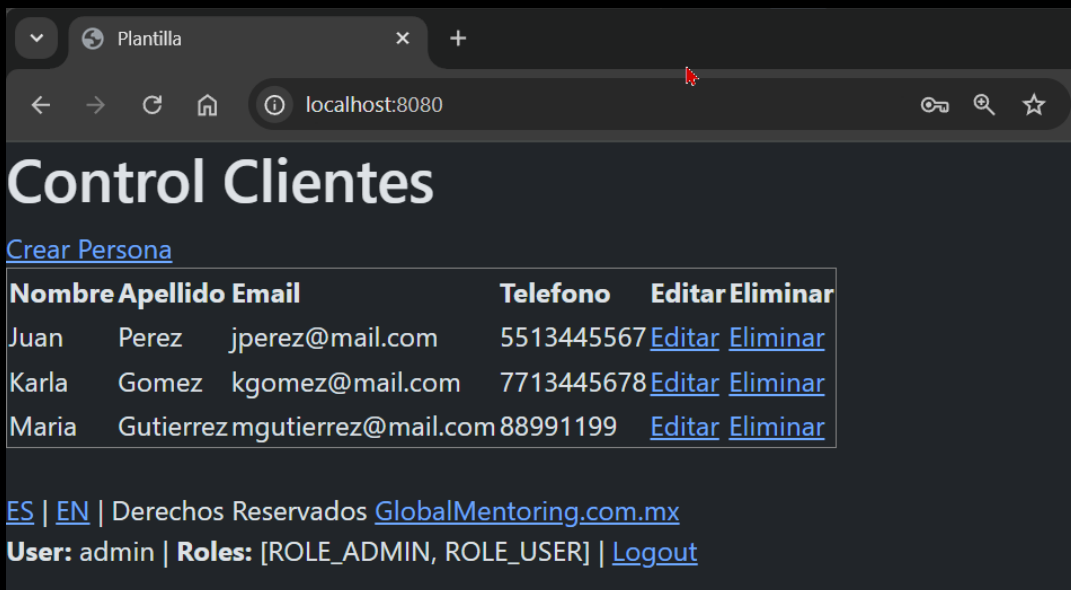
localhost:8080/login

Login

Username:

Password:

Login



Plantilla

localhost:8080

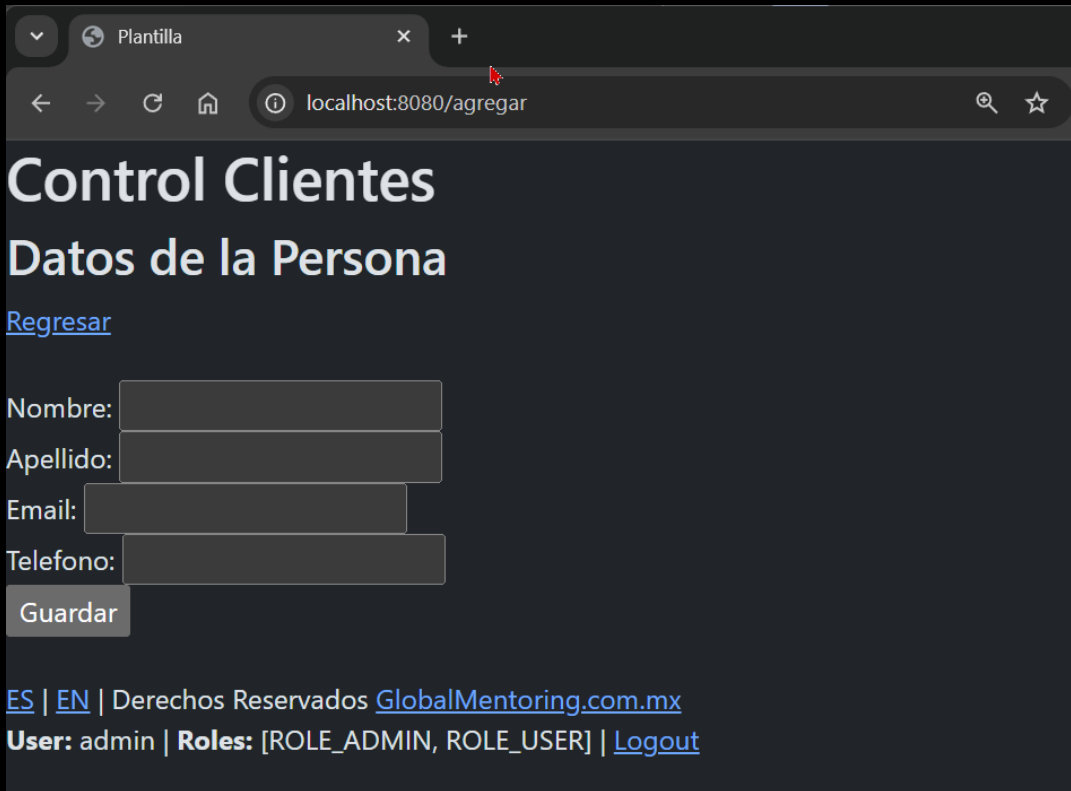
Control Clientes

[Crear Persona](#)

Nombre	Apellido	Email	Telefono	Editar	Eliminar
Juan	Perez	jperez@mail.com	5513445567	Editar	Eliminar
Karla	Gomez	kgomez@mail.com	7713445678	Editar	Eliminar
Maria	Gutierrez	mgutierrez@mail.com	88991199	Editar	Eliminar

[ES](#) | [EN](#) | Derechos Reservados [GlobalMentoring.com.mx](#)

User: admin | Roles: [ROLE_ADMIN, ROLE_USER] | [Logout](#)



The screenshot shows a web browser window with the address bar displaying 'localhost:8080/agregar'. The page title is 'Control Clientes' and the main heading is 'Datos de la Persona'. There is a link 'Regresar' and a form with four input fields: 'Nombre:', 'Apellido:', 'Email:', and 'Telefono:'. Below the form is a 'Guardar' button. At the bottom, there is a footer with links 'ES | EN |', copyright information 'Derechos Reservados GlobalMentoring.com.mx', user information 'User: admin | Roles: [ROLE_ADMIN, ROLE_USER]', and a 'Logout' link.

Conclusión

En esta lección:

- ✓ Aprendimos a integrar correctamente Bootstrap y FontAwesome usando WebJars en nuestro proyecto Spring Boot.
- ✓ Organizamos nuestro layout utilizando fragmentos (`th:fragment`) para facilitar el mantenimiento.
- ✓ Configuramos la clase SecurityConfig para aceptar las peticiones de webjars y así aceptar las peticiones de archivos .css y .js de Bootstrap y FontAwesome.
- ✓ Dejamos la base lista para mejorar las páginas visualmente en las próximas lecciones.

Ahora nuestra aplicación ya está preparada para trabajar con estilos profesionales utilizando Bootstrap.



Código Final de cada archivo trabajado

 plantilla.html

```

<!DOCTYPE html>
<html data-bs-theme="dark"
  xmlns:th="http://www.thymeleaf.org"
  xmlns:sec="http://www.thymeleaf.org/extras/spring-security">
  <head th:fragment="head">
    <title>Plantilla</title>
    <meta charset="UTF-8"/>
    <!-- CSS de Bootstrap -->
    <link rel="stylesheet" th:href="@{/webjars/bootstrap/css/bootstrap.min.css}"/>
    <!-- CSS de Font Awesome -->
    <link rel="stylesheet" th:href="@{/webjars/font-awesome/css/all.css}"/>
    <!-- JavaScript de Bootstrap (incluye Popper.js) -->
    <script th:src="@{/webjars/bootstrap/js/bootstrap.bundle.min.js}"></script>
  </head>
  <body>
    <header th:fragment="header">
      <h1 th:text="#{plantilla.titulo}">Titulo</h1>
    </header>

    <footer th:fragment="footer">
      <div>
        <br/>
        <a th:href="@{/ (lang=es)}">ES</a> |
        <a th:href="@{/ (lang=en)}">EN</a> |
        <span>
          [[#{plantilla.pie-pagina}]]
          <a href="http://www.globalmentoring.com.mx"
            target="_blank">GlobalMentoring.com.mx</a>
        </span>
        <form method="POST" th:action="@{/logout}">
          <b>User:</b> <span sec:authentication="name">Usuario que hizo login</span> |
          <b>Roles:</b> <span sec:authentication="principal.authorities">Roles</span> |
          <a href="#" onclick="this.parentNode.submit();">Logout</a>
        </form>
      </div>
    </footer>

  </body>
</html>

```

index.html

```

<!DOCTYPE html>
<html data-bs-theme="dark"

```



```

xmlns:th="http://www.thymeleaf.org"
xmlns:sec="http://www.thymeleaf.org/extras/spring-security">
<head th:replace=~{layout/plantilla :: head}>
    <title>Inicio</title>
    <meta charset="UTF-8">
</head>
<body>

    <header th:replace=~{layout/plantilla :: header}></header>

    <a sec:authorize="hasRole('ROLE_ADMIN')" th:href="@{/agregar}">[[#{persona.crear}]]</a>

    <div th:if="${personas != null and !personas.empty}">
        <table border="1">
            <tr>
                <th>[[#{persona.nombre}]]</th>
                <th>[[#{persona.apellido}]]</th>
                <th>[[#{persona.email}]]</th>
                <th>[[#{persona.telefono}]]</th>
                <th sec:authorize="hasRole('ROLE_ADMIN')">[[#{accion.editar}]]</th>
                <th sec:authorize="hasRole('ROLE_ADMIN')">[[#{accion.eliminar}]]</th>
            </tr>
            <tr th:each="persona : ${personas}">
                <td th:text="${persona.nombre}">Nombre</td>
                <td th:text="${persona.apellido}">Apellido</td>
                <td th:text="${persona.email}">Email</td>
                <td th:text="${persona.telefono}">Telefono</td>
                <td sec:authorize="hasRole('ROLE_ADMIN')">
                    <a th:href="@{/editar/{id}(id=${persona.idPersona})}">[[#{accion.editar}]]</a>
                </td>
                <td sec:authorize="hasRole('ROLE_ADMIN')">
                    <a
                        th:href="@{/eliminar(idPersona=${persona.idPersona})}">[[#{accion.eliminar}]]</a>
                </td>
            </tr>
        </table>
    </div>

    <div th:if="${personas == null or personas.empty}">
        [[#{persona.lista-vacia}]]
    </div>

    <footer th:replace=~{layout/plantilla :: footer}></footer>

</body>

```

</html>

login.html

```
<!DOCTYPE html>
<html data-bs-theme="dark"
  xmlns:th="http://www.thymeleaf.org">
  <head th:replace="~{layout/plantilla :: head}">
    <title>Login</title>
    <meta charset="UTF-8"/>
  </head>
  <body>
    <h1>Login</h1>

    <form method="POST" th:action="@{/login}">
      <label for="username">Username:</label>
      <input type="text" name="username" id="username"/>
      <br/>
      <label for="password">Password:</label>
      <input type="password" name="password" id="password"/>
      <br/>
      <input type="submit" value="Login"/>
    </form>

  </body>
</html>
```

modificar.html

```
<!DOCTYPE html>
<html lang="es" data-bs-theme="dark"
  xmlns:th="http://www.thymeleaf.org">
  <head th:replace="~{layout/plantilla :: head}">
    <title>Datos de la Persona</title>
    <meta charset="UTF-8"/>
  </head>
  <body>
    <header th:replace="~{layout/plantilla :: header}"></header>

    <h2>[[#{persona.formulario}]]</h2>
    <a th:href="@{/}">[[#{accion.regresar}]]</a>
    <br/>
    <form th:action="@{/guardar}" method="post" th:object="${persona}">
      <input type="hidden" name="idPersona" th:field="*{idPersona}"/>
```

```

<br/>
<label for="nombre">[[#{persona.nombre}]]:</label>
<input type="text" name="nombre" th:field="*{nombre}"/>
<span th:if="${#fields.hasErrors('nombre')}}" th:errors="*{nombre}">Error Nombre</span>
<br/>
<label for="apellido">[[#{persona.apellido}]]:</label>
<input type="text" name="apellido" th:field="*{apellido}"/>
<span th:if="${#fields.hasErrors('apellido')}}" th:errors="*{apellido}">Error Apellido</span>
<br/>
<label for="email">[[#{persona.email}]]:</label>
<input type="text" name="email" th:field="*{email}"/>
<span th:if="${#fields.hasErrors('email')}}" th:errors="*{email}">Error Email</span>
<br/>
<label for="telefono">[[#{persona.telefono}]]:</label>
<input type="tel" name="telefono" th:field="*{telefono}"/>
<br/>
<input type="submit" name="guardar" th:value="#{accion.guardar}"/>
</form>

<footer th:replace="~{layout/plantilla :: footer}"></footer>
</body>
</html>

```

SecurityConfig.java

```

package gm.web;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {

    private final UserDetailsService userDetailsService;

    public SecurityConfig(UserDetailsService userDetailsService) {
        this.userDetailsService = userDetailsService;
    }
}

```

```

@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    return http
        .authorizeHttpRequests(authorize -> authorize
            .requestMatchers("/webjars/**", "/resources/**", "/login").permitAll()
            .requestMatchers("/editar/**", "/agregar/**", "/eliminar/**").hasRole("ADMIN")
            .requestMatchers("/").hasAnyRole("USER", "ADMIN")
            .anyRequest().authenticated()
        )
        .formLogin(form -> form
            .loginPage("/login")
            .defaultSuccessUrl("/", true)
            .permitAll()
        )
        .logout(logout -> logout
            .logoutUrl("/logout")
            .logoutSuccessUrl("/login?logout")
            .invalidateHttpSession(true)
            .deleteCookies("JSESSIONID")
        )
        .exceptionHandling(exception -> exception
            .accessDeniedPage("/errores/403")
        )
        .build();
}

@Bean
public DaoAuthenticationProvider authenticationProvider() {
    DaoAuthenticationProvider authProvider = new DaoAuthenticationProvider();
    authProvider.setUserDetailsService(userDetailsService);
    authProvider.setPasswordEncoder(passwordEncoder()); // Aquí llamamos al método, no al atributo
    return authProvider;
}

@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
}

```

Sigue adelante con tu aprendizaje 🚀, ¡el esfuerzo vale la pena!

¡Saludos! 🙌

Ing. Marcela Gamiño e Ing. Ubaldo Acosta

Fundadores de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)